

Growth of Functions

	<i>constant</i>	<i>logarithmic</i>	<i>linear</i>	<i>N-log-N</i>	<i>quadratic</i>	<i>cubic</i>	<i>exponential</i>
<i>n</i>	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(n^3)$	$O(2^n)$
1	1	1	1	1	1	1	2
2	1	1	2	2	4	8	4
4	1	2	4	8	16	64	16
8	1	3	8	24	64	512	256
16	1	4	16	64	256	4,096	65536
32	1	5	32	160	1,024	32,768	4,294,967,296
64	1	6	64	384	4,069	262,144	1.84×10^{19}

Asymptotic Notations

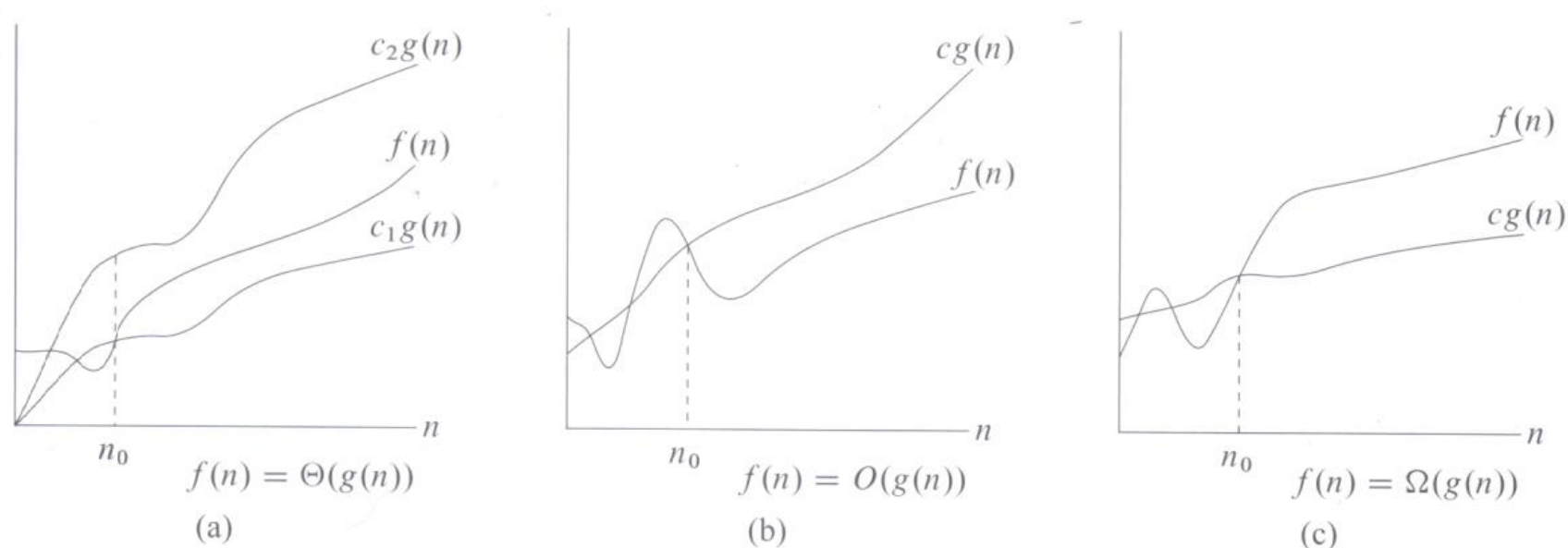
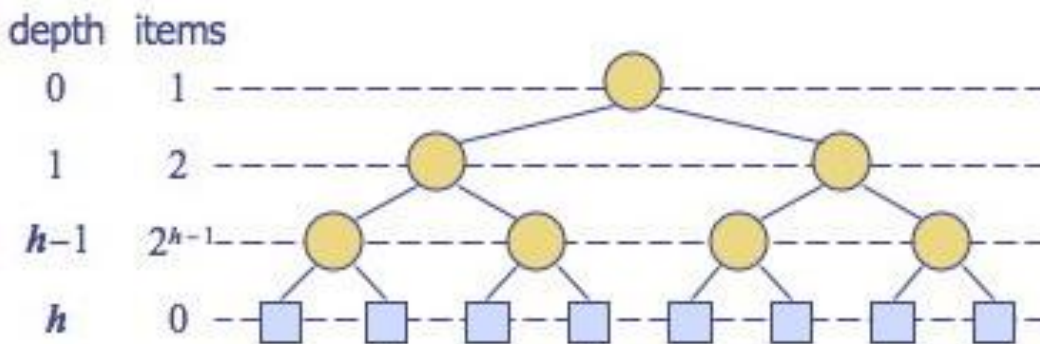


Figure 3.1 Graphic examples of the Θ , O , and Ω notations. In each part, the value of n_0 shown is the minimum possible value; any greater value would also work. **(a)** Θ -notation bounds a function to within constant factors. We write $f(n) = \Theta(g(n))$ if there exist positive constants n_0 , c_1 , and c_2 such that at and to the right of n_0 , the value of $f(n)$ always lies between $c_1g(n)$ and $c_2g(n)$ inclusive. **(b)** O -notation gives an upper bound for a function to within a constant factor. We write $f(n) = O(g(n))$ if there are positive constants n_0 and c such that at and to the right of n_0 , the value of $f(n)$ always lies on or below $cg(n)$. **(c)** Ω -notation gives a lower bound for a function to within a constant factor. We write $f(n) = \Omega(g(n))$ if there are positive constants n_0 and c such that at and to the right of n_0 , the value of $f(n)$ always lies on or above $cg(n)$.

Binary Tree, Summations...

■ Binary Tree



■ Summations

$$\text{Let } s = a + ar + ar^2 + ar^3 + \cdots + ar^{n-1}.$$

$$\text{Then } rs = ar + ar^2 + ar^3 + ar^4 + \cdots + ar^n$$

$$\text{Then } s - rs = a - ar^n$$

$$\text{Then } s(1 - r) = a(1 - r^n), \text{ so } s = a \frac{1 - r^n}{1 - r} \quad (\text{if } r \neq 1).$$

Sums of Powers of Natural Numbers

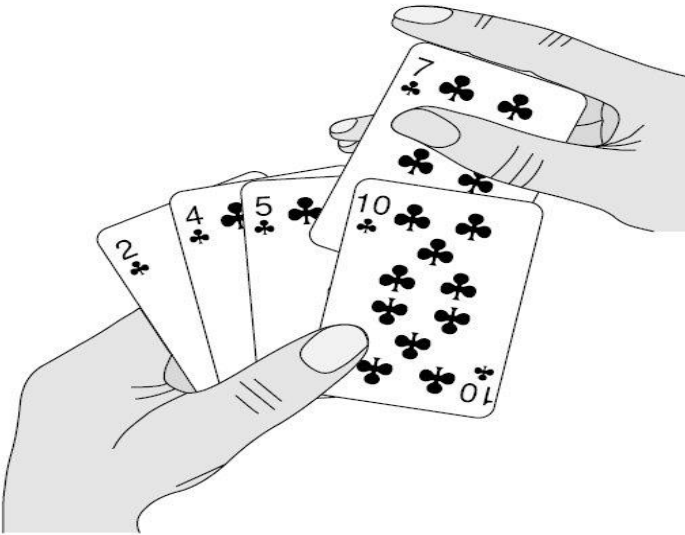
$$1. \quad 1 + 2 + 3 + \cdots + n = \sum_{i=1}^n i = \frac{n(n+1)}{2}$$

$$2. \quad 1^2 + 2^2 + 3^2 + \cdots + n^2 = \sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$3. \quad 1^3 + 2^3 + 3^3 + \cdots + n^3 = \sum_{i=1}^n i^3 = \frac{n^2(n+1)^2}{4}$$

Sorting Algorithms

Insertion sort



INSERTION-SORT(A)		<i>cost</i>	<i>times</i>
1	for $j = 2$ to $A.length$	c_1	n
2	$key = A[j]$	c_2	$n - 1$
3	// Insert $A[j]$ into the sorted sequence $A[1..j - 1]$.	0	$n - 1$
4	$i = j - 1$	c_4	$n - 1$
5	while $i > 0$ and $A[i] > key$	c_5	$\sum_{j=2}^n t_j$
6	$A[i + 1] = A[i]$	c_6	$\sum_{j=2}^n (t_j - 1)$
7	$i = i - 1$	c_7	$\sum_{j=2}^n (t_j - 1)$
8	$A[i + 1] = key$	c_8	$n - 1$

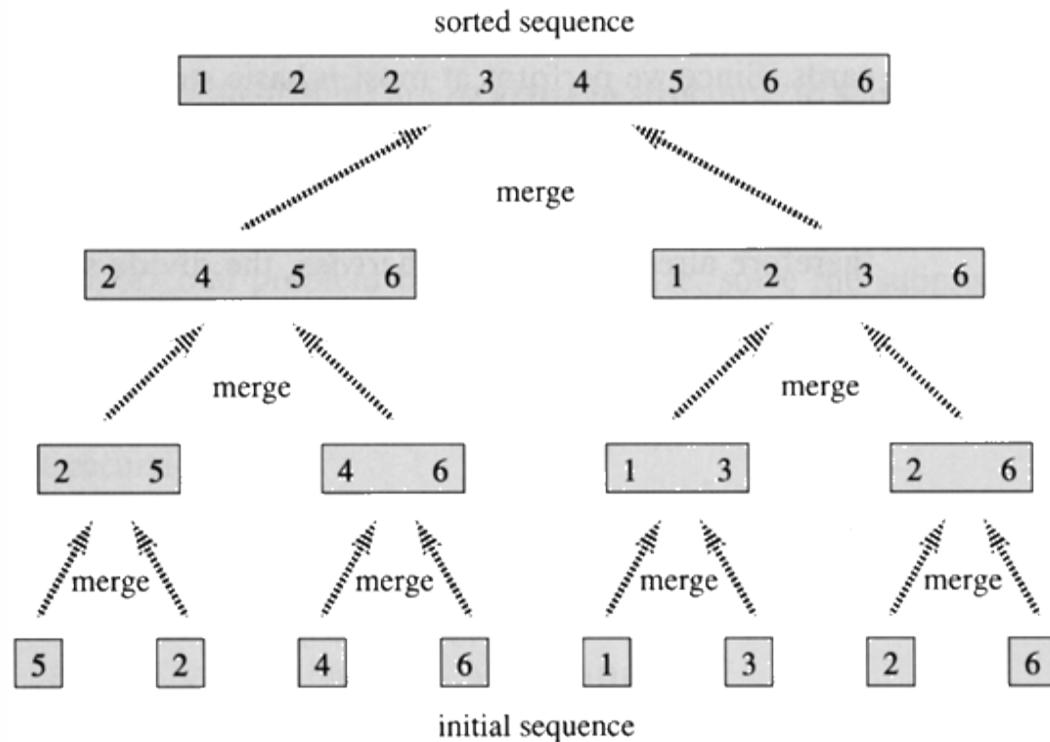
Note: More Sorting algorithms (Bubble sort, selection sort) are covered in the class, take notes in class

Divide and Conquer: Merge Sort

MERGE(A, p, q, r)

```

1   $n_1 = q - p + 1$ 
2   $n_2 = r - q$ 
3  let  $L[1..n_1 + 1]$  and  $R[1..n_2 + 1]$  be new arrays
4  for  $i = 1$  to  $n_1$ 
5       $L[i] = A[p + i - 1]$ 
6  for  $j = 1$  to  $n_2$ 
7       $R[j] = A[q + j]$ 
8   $L[n_1 + 1] = \infty$ 
9   $R[n_2 + 1] = \infty$ 
10  $i = 1$ 
11  $j = 1$ 
12 for  $k = p$  to  $r$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 
    
```



MERGE-SORT(A, p, r)

```

1  if  $p < r$ 
2       $q = \lfloor (p + r) / 2 \rfloor$ 
3      MERGE-SORT( $A, p, q$ )
4      MERGE-SORT( $A, q + 1, r$ )
5      MERGE( $A, p, q, r$ )
    
```

Divide and Conquer: Quick Sort

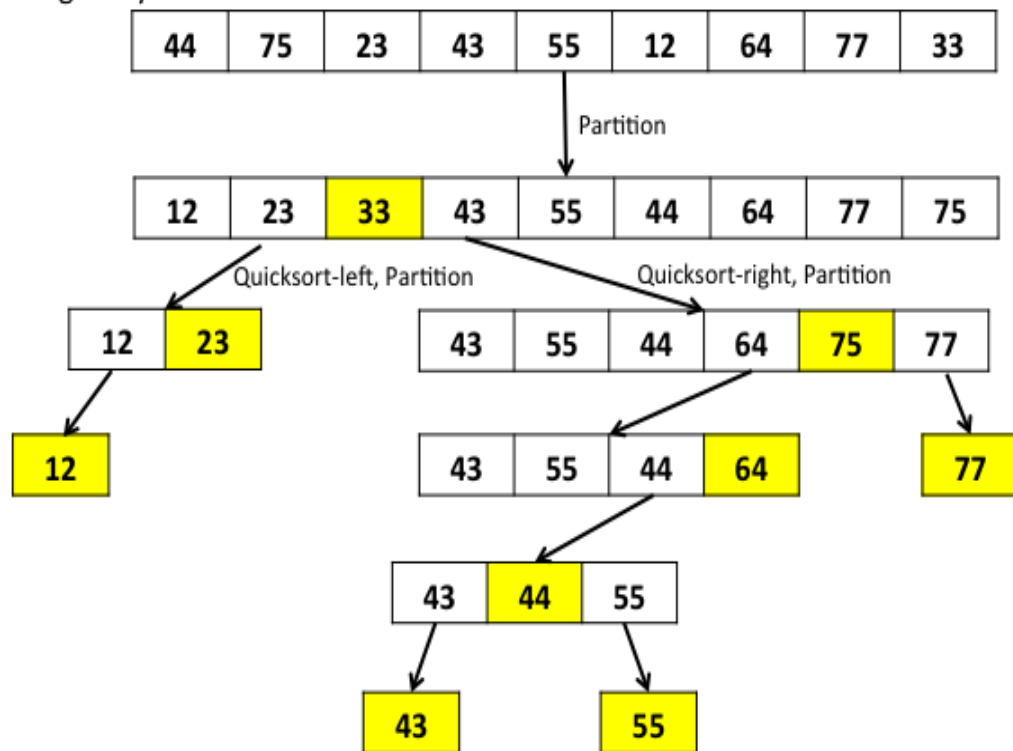
QUICKSORT(A, p, r)

```
1  if  $p < r$ 
2     $q = \text{PARTITION}(A, p, r)$ 
3    QUICKSORT( $A, p, q - 1$ )
4    QUICKSORT( $A, q + 1, r$ )
```

PARTITION(A, p, r)

```
1   $x = A[r]$ 
2   $i = p - 1$ 
3  for  $j = p$  to  $r - 1$ 
4    if  $A[j] \leq x$ 
5       $i = i + 1$ 
6      exchange  $A[i]$  with  $A[j]$ 
7  exchange  $A[i + 1]$  with  $A[r]$ 
8  return  $i + 1$ 
```

Starting array



Resulting array

12	23	33	43	44	55	64	75	77
----	----	----	----	----	----	----	----	----

Masters Theorem

The Master Theorem

- if $T(n) = aT(n/b) + f(n)$ then

$$T(n) = \left\{ \begin{array}{ll} \Theta(n^{\log_b a}) & f(n) = O(n^{\log_b a - \varepsilon}) \\ \Theta(n^{\log_b a} \log n) & f(n) = \Theta(n^{\log_b a}) \\ \Theta(f(n)) & f(n) = \Omega(n^{\log_b a + \varepsilon}) \text{ AND} \\ & af(n/b) < cf(n) \text{ for large } n \end{array} \right\} \begin{array}{l} \varepsilon > 0 \\ c < 1 \end{array}$$